



BACHELORARBEIT

USING A LARGE LANGUAGE MODEL (LLM) TO BUILD A
KNOWLEDGE GRAPH FROM SCRATCH
TO IMPROVE RETRIEVAL-AUGMENTED GENERATION
(RAG)
FOR ANSWERING QUESTIONS ON CUSTOM DATA

Verfasser

Dragomir Marinov

angestrebter akademischer Grad

Bachelor of Science (BSc)

Wien, 2026

Studienkennzahl lt. Studienblatt: UA 033 526

Fachrichtung: Wirtschaftsinformatik

Betreuer: Dipl.-Ing. Dr.techn. Marian Lux

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	Retrieval-Augmented Generation	4
1.3	Problems with Standard RAG	5
1.4	Knowledge Graph as a Solution	6
2	Methodology	9
2.1	Knowledge Graph Application	9
2.1.1	Text Extraction	10
2.1.2	Chunking	11
2.1.3	Entity and Relationship Extraction	12
2.1.4	Storage in MongoDB	14
2.1.5	Entity management	14
2.1.6	Graph visualization	15
2.2	Integration Strategy	17
2.2.1	Surveyed Strategies	17
2.2.2	Chosen Approach	19
2.3	RAG Chatbot	20
2.3.1	Entity Extraction	21
2.3.2	Fuzzy Matching	22
2.3.3	Database retrieval	23
2.3.4	Prompt Design	24
2.4	Complete System Workflow	25
3	Evaluation	26
3.1	Evaluation Iteration 1 - Local LLM Judge	26
3.1.1	Problems Identified and Changes Made	27
3.1.2	Limitations	28
3.2	Evaluation Iteration 2 - External LLM Judge	28
3.2.1	Results	28
3.3	Evaluation Iteration 3 - Multi-Document Evaluation	30
3.3.1	Results	30
3.3.2	Cross-Document Analysis	31
3.4	Discussion	31
4	Conclusion and Future Work	32
4.1	Conclusion	32
4.2	Limitations	33
4.3	Future Work	33

Abstract

Large Language Models (LLMs) have demonstrated strong capabilities in natural language understanding and generation. However, their knowledge is limited to the data available during training, making them unsuitable for scenarios that require access to private or domain-specific information. Retrieval-Augmented Generation (RAG) [15] addresses this limitation by dynamically retrieving relevant document chunks and adding them to the prompt provided to the model. Nevertheless, the standard RAG approach still faces some challenges: retrieved chunks can be semantically homogeneous and redundant [31], relationships between entities distributed across multiple chunks are not explicitly represented [20] and hallucinations may still occur when relevant information is not successfully retrieved[27].

This bachelor’s thesis presents a combination of a standard RAG with a custom-built Knowledge Graph to address the limitations of RAG and improve its answers by including relationships between retrieved data chunks and entities. The main focus is the development of a Knowledge Graph Application from scratch that extracts named entities with descriptions and the relationships between them from uploaded documents. Furthermore, a RAG chatbot is implemented that combines the data from the Knowledge Graph with its chunk data from a vector database to generate better answers. All components of the applications run locally using Llama 3.1:8B through Ollama [19], allowing the whole system to run on consumer hardware. MongoDB is used for knowledge graph storage and ChromaDB for vector storage.

An evaluation of the chatbot answers is carried out using an LLM-as-a-judge methodology [30] in three iterations. First, a custom tool was built to test the system on a single document using the local LLM model, which judged the answers based on predefined ground truths. This was afterwards replaced with the more capable Claude Sonnet [2], which verified answers directly against the source documents. Finally, more documents were added to test whether the system could connect information across multiple sources. The evaluation confirmed that the knowledge graph augmentation provides measurable improvements specifically for queries that require the connection of information across multiple documents, which is the case the knowledge graph was designed to address. This demonstrates that such benefits can be achieved even with relatively small local language model such as Llama 3.1:8B.

1 Introduction

1.1 Background and Motivation

The rapid development of Large Language Models (LLMs) has transformed the interaction with information systems. Models like GPT-4, Claude and the Llama family can generate meaningful texts, understand complex language and perform advanced reasoning. However, their knowledge is limited to the static datasets used during training and therefore only extends to the information available at the time of training. Thus, they are unaware of private documents or domain-specific information that was not present during training. This is an important constraint for the real-world use of LLMs.

For example, if a researcher asks an LLM about an internal report, a company queries information about its own products, or a student asks about a recently published article, the generated answers rely solely on the parametric knowledge of the model. As a result, the answers may be outdated, incomplete, or inaccurate if the model’s training data do not fully cover the topic of the question. Furthermore, the LLMs can provide answers that appear to be correct but are not supported by the available information, which is called hallucination [27].

Therefore, allowing LLMs to work on custom user documents without fine-tuning is one of the key challenges in NLP today.

1.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. [15] in 2020, has become one of the most widely adopted solutions to this limitation. RAG enables the model to retrieve relevant information from an external document store at query time and include it into its response, instead of depending just on the parametric knowledge stored within the model weights. This allows LLMs to answer questions based on user-provided document collections without requiring modifications to the underlying model parameters.

A standard RAG pipeline consists of two main phases. In the first phase, the **indexing phase**, documents are split into smaller text segments called chunks. Each chunk is converted into a dense vector representation (embedding) using an embedding model, and these embeddings are then stored in a vector database. During the second phase, the **retrieval and generation phase**, the user’s query is converted into an embedding using the same embedding model. A similarity search is performed against the stored chunk embeddings, and the top- k most similar chunks are retrieved. These chunks, together with the original query, are provided as additional context to the LLM prompt, allowing the model to generate a response grounded in the retrieved information. Exactly this process is shown in Figure 1.

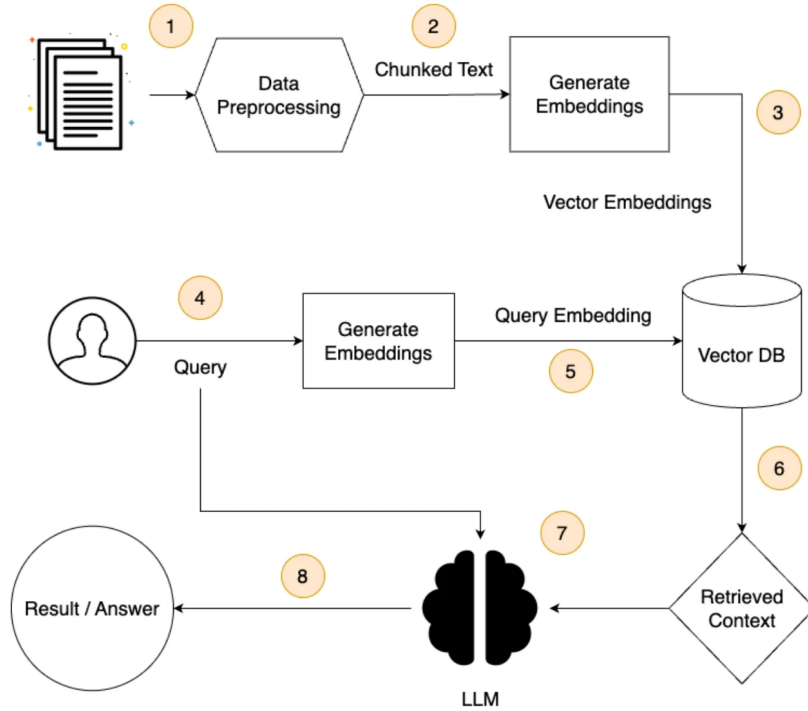


Figure 1: A Standard RAG pipeline. Source: Jain [10].

Steps 1-3 show the indexing phase, including document pre-processing, embedding generation, and vector storage. Steps 4-8 represent the query phase: the user submits a query (4), which is converted into an embedding (5) and used to retrieve the most similar chunks from the vector database (6). The retrieved context and the query are passed to the LLM (7), which generates the final response (8).

The main advantage of RAG is its flexibility, as new knowledge can be incorporated by updating the external document collection instead of modifying or retraining the underlying model. The relatively straightforward integration of RAG with existing LLM systems has contributed to its increasing adoption. Gao et al. [7] provide a comprehensive survey showing the widespread adoption of RAG for knowledge-intensive question answering over private document collections. Furthermore, Lewis et al. [15] demonstrated that retrieval augmentation improves factual generation and performance on knowledge-intensive tasks compared to purely parametric generation.

1.3 Problems with Standard RAG

Despite its widespread adoption, standard RAG still has some weaknesses that limit its effectiveness on complex queries requiring reasoning across multiple

sources.

Chunk homogeneity. Since the retrieval is primarily driven by semantic similarity rather than information diversity, the top- k retrieved chunks can sometimes be semantically redundant. Multiple chunks contain similar information expressed in slightly different ways, providing the model with variations of a single narrow aspect instead of a broader and more informative context [31].

No relational context. Text chunks are retrieved as isolated fragments without an explicit representation of relationships between entities across documents. Standard RAG does not maintain knowledge about how different entities, concepts, or facts are connected. Consequently, connecting information from multiple sources depends on whether the relevant chunks are retrieved together and whether the LLM can infer the underlying relationships from the provided context. As the document collection grows, the probability of retrieving all relevant information within the same context window decreases [20].

Hallucination. Although RAG can reduce hallucinations by grounding responses in retrieved information, it does not eliminate them entirely. Hallucinations may still occur when the retrieved context is incomplete, ambiguous, or does not directly address the question. LLM may attempt to fill missing information using its parametric knowledge, which can lead to plausible-sounding but incorrect responses [27].

1.4 Knowledge Graph as a Solution

A Knowledge Graph (KG) is a structured representation of information in the form of a directed graph. It consists of nodes, which represent named entities and labelled edges, which represent the relationships between them [20, 11]. Each relationship is represented in the form of a subject-predicate-object triple, where the subject and the object are nodes and the predicate describes their connection. Entities can also carry descriptive attributes that provide richer context beyond the triple.

Figure 2 illustrates this structure using an example knowledge graph over the AI domain.

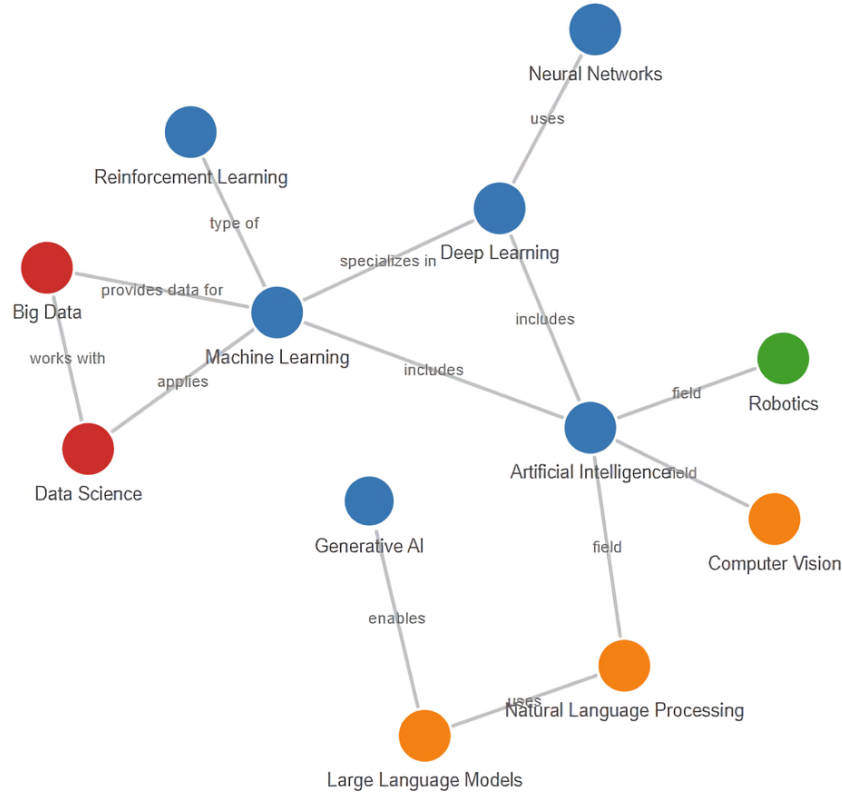


Figure 2: A knowledge graph over the AI domain. Source: Andrés [1].

The nodes *Machine Learning* and *Deep Learning* are connected through the edge *specializes in*, forming the triple $\{Machine\ Learning, specializes\ in, Deep\ Learning\}$. Similarly, the triple $\{Large\ Language\ Models, uses, Natural\ Language\ Processing\}$ represents a relationship between these two concepts. Note that the relationships here are not directed, but on some graphs they can be directed to give more information about the flow of the connection.

This graph structure addresses all three weaknesses of standard RAG discussed in Section 1.3. Chunk redundancy can be reduced because the system can retrieve structured entity information and relationships in addition to semantically similar text chunks. Furthermore, relationships between entities are stored as edges and can be retrieved just by the entity name, independently of whether the corresponding text chunks appear together among the top- k retrieved results. Zhu et al. [31] support this idea by showing that extending RAG with a knowledge graph can reduce redundancy and improve connectivity by providing structured, non-overlapping information alongside extracted chunks.

Regarding hallucination, Pan et al. [20] demonstrate across multiple empirical studies that connecting LLM outputs to verifiable knowledge graph facts

can improve factual consistency, since the model uses verifiable facts instead of only similarity-ranked text.

A research across several other papers has also demonstrated the benefits of KG-augmented retrieval. Edge et al.[6] showed in their GraphRAG system that graph-based retrieval improves responses for thematic and cross-document questions compared to vector-only retrieval. Baek et al.[4] demonstrated that enriching prompts with relevant KG triples improves factual accuracy on knowledge-intensive questions. Similarly, Soman et al.[25] reported improvements in the biomedical domain, where prompts enriched with entities and relationships from a domain-specific knowledge graph led to more accurate answers.

Together, these studies indicate that the benefit of Knowledge Graph augmentation is not limited to one domain or one retrieval style. It recurs across different implementations of the same underlying idea of supplying the model with explicit, verifiable relational facts. The construction of such graphs from raw text using LLMs is itself an active research area, surveyed by Xu et al.[29] and Kommineni et al.[12].

Based on this literature, the next section describes the methodology and approach developed for this thesis.

2 Methodology

The main objective of the project covered in this thesis is to build a Knowledge Graph from scratch from user-provided documents from any domain. Additionally, the knowledge graph should be then used in a Retrieval-Augmented Generation chatbot and attempt to improve its answering abilities. To achieve this goal, two stand-alone applications were developed. This section focuses on the design and implementation of the them, explaining the decisions taken and strategies chosen. The first application is a Knowledge Graph Application, that automatically constructs a knowledge graph from user-provided documents by extracting entities, their descriptions, and the relationships between them. Besides creating the graph, it also provides an interface for visualizing, editing, and managing the extracted knowledge. The second application is an RAG-based chatbot that combines the data stored in the Knowledge Graph with a vector-based document retrieval to answer user questions. Another important detail covered in this section is the integration strategy that connects the knowledge graph to the RAG pipeline.

Python 3.11 was chosen as the implementation language because it is the dominant language for artificial intelligence and data engineering. Its ecosystem provides libraries for document processing, large language models, vector databases, and web application development. A design goal of the project was to ensure that the entire system operates locally. Therefore, all components run on standard hardware with at least 8 GB of RAM.

The language model used throughout both applications is Llama 3.1:8B, ran locally through Ollama [19]. Within the Knowledge Graph Application, Llama 3.1:8B is responsible for extracting entities, descriptions, and relationships from the uploaded documents, while in the Chatbot Application it generates responses based on the retrieved context. The model was chosen because its 8-billion parameter size fits within the hardware requirements.

The project also uses two local databases. MongoDB stores the entities and relationships that form the Knowledge Graph, whereas ChromaDB stores the vector embeddings required by the RAG pipeline. Their data structures and usage are described in the corresponding application sections later in this chapter.

2.1 Knowledge Graph Application

The Knowledge Graph Application creates a structured knowledge graph from user-provided documents by extracting named entities, their descriptions, and the relationships between them as subject-predicate-object triples. It allows users to search for entities, edit them and their descriptions, and create, modify, or remove relationships. This way the user has full control of the data. Furthermore, the application provides interactive visualizations of both the complete knowledge graph and entity-specific subgraphs, making it easier to explore and understand the stored information.

The workflow of the knowledge graph creation consists of 5 stages illustrated in Figure 3.

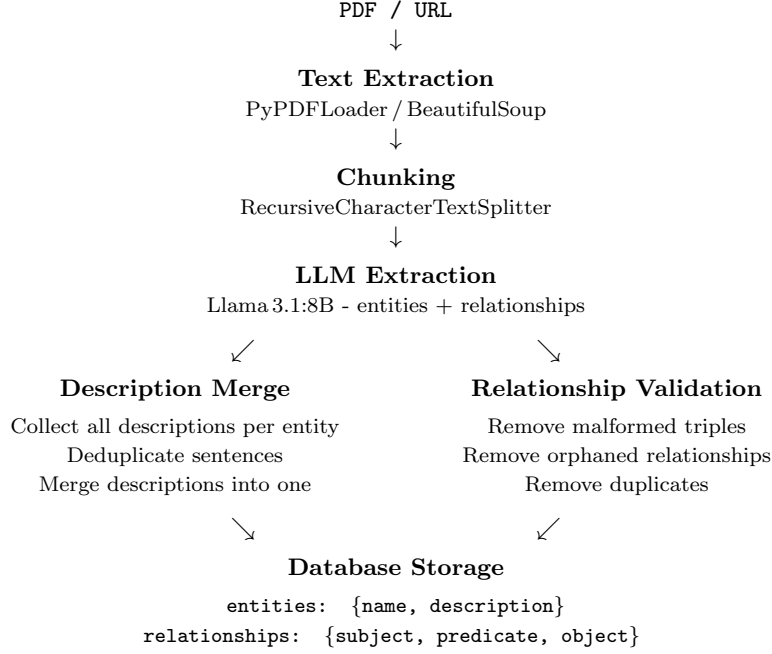


Figure 3: The Knowledge Graph construction pipeline.

The process begins with text extraction and chunking, proceeds through LLM-based entity and relationship extraction, applies post-processing in two parallel steps (description merge and relationship validation), and ends with storage in a database.

2.1.1 Text Extraction

Text Extraction is the first step of the knowledge graph construction and begins right after the user uploads the documents. The application accepts two input types: PDF documents and web page URLs, from which text can be extracted.

PDF documents. Text is extracted from PDFs using **PyPDFLoader** from the LangChain community library [14], which supports multi-page documents and preserves their paragraph structure. The extracted text is then passed directly to the chunking stage described in Section 2.1.2.

Web page URLs. Extracting meaningful content from web pages is more complex than processing PDF documents because HTML pages contain significant amounts of additional information, such as navigation menus, advertising, footers, scripts, and captions, which must be removed before processing. Therefore, a custom extraction logic was implemented. It first identifies the primary

content area using a fallback strategy, searching for a `<main>` element first, then an `<article>` element and finally it falls back to `<body>` if neither is found. This handles the variety of page structures on different websites. After locating the main content, non-informative elements like figures, footers and scripts are removed. The remaining text is collected only from semantic content elements, including paragraphs, headings, and list items. Non-paragraph tags have punctuation appended if missing to ensure that the text consists of coherent sentences.

Before chunking, the extracted text undergoes an additional cleaning step specific to web content. Duplicate sentences are removed, author and date bylines are stripped, duplicate adjacent words caused by HTML rendering artifacts are corrected, and whitespace is normalized. Sentences shorter than 15 characters are discarded as they do not carry meaningful information.

2.1.2 Chunking

The cleaned text is split into chunks using LangChain’s `RecursiveCharacterTextSplitter` [14]. The splitter divides the text using a prioritized list of separators such as paragraphs, sentences, and spaces until the target chunk size is reached, while maintaining context and preventing sentences from breaking apart. The configuration used in the application is shown below.

Listing 1: Shared text splitter configuration.

```
text_splitter = RecursiveCharacterTextSplitter(  
    chunk_size=1200,  
    chunk_overlap=200,  
    separators=["\n\n", ". ", "! ", "? ", "; ", "\n", " "],  
    add_start_index=False  
)
```

These configuration parameters were determined through empirical testing during development. Different chunk sizes and overlap values were evaluated to find a balance between extraction quality and processing efficiency. The final configuration was selected based on its ability to provide sufficient context for reliable entity and relationship extraction while avoiding unnecessary redundancy.

The **chunk size of 1200 characters** was chosen to give the LLM enough context to understand the entities and relationships within each chunk while keeping the content focused. Chunks that are too short risk incomplete entity descriptions and chunks that are too long risk producing noisy extractions where the model picks entities from unrelated topics that happen to appear together or totally miss out an important entity.

The **overlap of 200 characters** means that the end of one chunk and the beginning of the next chunk share 200 characters of text. This is important, since a relationship between two entities does not always fit within one chunk. Without overlap, such a relationship would be split and neither chunk would

contain enough information for the LLM to extract it. The value of 200 was found to be large enough to cover a typical sentence boundary while small enough to avoid redundancy.

The **separator hierarchy** reflects natural text structure: the splitter first attempts to split at double newlines (paragraph boundaries), then at sentence-ending punctuation, then at semicolons, then at single newlines, and finally at spaces as a last resort. This way splits occur at linguistically meaningful places wherever possible.

The quality of the extracted entities and relationships depends heavily on the chunking strategy, since each chunk is processed independently by the language model during the extraction stage.

2.1.3 Entity and Relationship Extraction

Before extraction begins, chunks from all uploaded documents are collected into a single list. The idea behind this is to process all the documents together rather than one at a time. This allows description merging to operate across the entire document set at once, so richer entity descriptions are being produced by combining information from all sources.

Extraction prompt. Each chunk is passed to Llama 3.1:8B with a structured prompt that asks the model to perform two tasks simultaneously. First one is to identify named entities and provide a short description of each in the context of the document, and the other one is to find relationships between those entities and form subject-predicate-object triples. Both tasks are handled in a single LLM call per chunk to avoid doubling the inference time.

The core explicit instructions given to the model are as follows:

1. Entity descriptions must be detailed and only include information explicitly present in the text. Do not assume, infer, or add facts not stated in the text. (10 words minimum, strictly based on text)
2. Use consistent entity names throughout (prefer full names over nicknames)
 - 2.1. if an entity name is a synonym or abbreviation for another entity name make a relationship showing it
3. Extract 12-15 entities maximum for quality over quantity
4. Extract 15-20 relationships that only reference entities in your entities dictionary
5. Ensure every relationship tuple has exactly 3 elements
6. If nicknames exist, mention them in the description but use the primary name as the key
7. keep the relation between subject entity and object entity short

8. Write only complete tuples, not tuples with less than three elements and no empty elements in the tuple

In addition to these instructions, the prompt contains validation rules as a reassurance of the expected result and a strict output format:

VALIDATION RULES:

- Every relationship subject and object MUST exist as a key in the entities dictionary
- Entity names should be normalized (proper capitalization, no articles)
- Relationship predicates should be specific and meaningful
- No incomplete tuples in relationships

Return format (EXACT structure required):

```
"entities":
  "Primary Entity Name": "Detailed description with
    context only from the text, including role,
    significance, and relevant details mentioned"
,
"relationships": [
  ("Subject Entity", "verb/phrase", "Object Entity")
]
```

These constraints are inspired by the findings of Xu et al.[29] and Kommineni et al. [12] that structured, constraint-rich prompts significantly improve the reliability of open LLM-based information extraction.

Description merging. Since the same entity can appear across many chunks of a document, the LLM produces multiple partial descriptions for the same entity name, each found in the local context of its chunk. These partial descriptions are merged into a single richer description through the following steps: First all descriptions for entities sharing the same name are collected. Then each one is split into sentences, which are normalized and deduplicated. The ones shorter than a minimum length are discarded and the remaining unique sentences are concatenated into one merged description. An entity appearing across ten chunks will therefore have its description built from the most informative sentences across all ten contexts, rather than being limited to what one chunk alone could capture.

Relationship validation. Relationships extracted by the LLM go through a validation and cleaning step before being stored. Three types of invalid triples are targeted for removal: malformed triples where the subject, predicate, or object is empty or missing; orphaned relationships where either the subject or

object does not correspond to an already extracted and stored entity; and exact duplicate triples. Only relationships that follow the predefined triple structure and consist of only known entities are being saved in the database.

2.1.4 Storage in MongoDB

After post-processing, the extracted entities and relationships are stored in a database.

A non-relational database was chosen because it stores data as JSON documents, which naturally match the entity and relationship objects produced by the extraction process. Each extracted entity is stored as a document containing a name and description, while each relationship is represented by a subject, predicate, and object. This document-based representation also simplifies storing, querying, and retrieving the knowledge graph. MongoDB [18] was selected as the database implementation. The use of a dedicated graph database, such as Neo4j, was not allowed, as the idea was to better understand how knowledge graph data can be modeled.

The database consists of two collections that represent the two fundamental components of a knowledge graph: entities and relationships [11]:

Table 1: MongoDB collection structure for knowledge graph storage.

Collection	Document structure
entities	<code>{"name": "Process Mining", "description": "A discipline at the intersection of data science and ..."}"</code>
relationships	<code>{"subject": "CustPro Project", "predicate": "applies", "object": "Process Mining"}"</code>

2.1.5 Entity management

The LLM can occasionally produce incomplete descriptions, redundant entities, or not clear enough relationships. Especially using a small model like Llama 3.1:8B. For this reason, the Knowledge Graph Application provides an interface for browsing, editing, and managing the extracted data, giving the user complete control over the graph after it is built. This also means the graph is not limited to what was extracted by the LLM, but the user can extend it by adding new entities and relationships manually, correct any errors and remove anything irrelevant to their domain.

The interface is built entirely using Streamlit [26], because it allows building a fully interactive web UI entirely in Python without requiring any HTML, CSS, or JavaScript. It provides many of the components required by both

applications, such as file uploaders, a chat interface, expandable context panels, and session state to persist data in UI interactions.

Particularly two pages are responsible for the entity management:

Browse page. The Browse Page provides an overview of all extracted entities in a grid, each displayed as a card with the entity name and a short preview of its description. A search bar allows the user to find specific entities. New entities can be added manually through a form on the page, and the entire graph data can be cleared by the user. Clicking on an entity opens a dedicated detail page that shows each entity individually, called Entity Detail Page.

Entity detail page. The Entity Detail Page grants more control over an individual entity and its relationships. The entity's name and description can be edited directly. All relationships involving the entity, whether it appears as the subject or the object, are displayed as subject-predicate-object triples and can be modified or deleted individually. New relationships can also be created by specifying a predicate and either selecting an existing entity as the object or creating a new one, which is then being added to the database.

Together, these two pages cover all CRUD (Create, Read, Update, Delete) operations for both entities and relationships. This gives the user full control over the extracted data, as he can not only correct inaccuracies introduced during the extraction process but also extend the knowledge graph with additional entities and relationships.

2.1.6 Graph visualization

The application also provides an interactive visualization of the full knowledge graph, rendered using `streamlit-agraph`, a Python wrapper around the `vis.js` graph visualization library. The user can zoom in and out, drag around, and select individual nodes. Clicking on a node highlights it in yellow and triggers a page reload.

The main challenge when building the visualization was the layout. Standard force-directed layouts tend to produce dense clusters in which many nodes overlap and make the graph difficult to read. Therefore, the node positions are computed in Python using `NetworkX` [8] before the graph is rendered. The computed layout is then preserved by disabling the physics simulation in `vis.js`.

The layout is designed to show the overall structure of the graph. Highly connected entities are placed near the center, while related entities are arranged around them. Entities belonging to smaller isolated components are placed to the right of the main graph, arranged as subgraphs and completely isolated nodes are placed above these graphs. When multiple relationships exist between the same pair of entities, they are combined into a single edge label and separated by “,” to reduce visual clutter.

Figure 4 shows a complete graph generated consisting of at least 100 nodes. The largest connected component is located on the left side, with smaller disconnected subgraphs arranged to the right. Note that this figure is intended to show the overall graph structure, so at this scale the node's and relationship's

labels are not shown. A closer look at individual entities and their connections is provided in the mini-graph section below (Figure 5).



Figure 4: View of a whole knowledge graph visualization.

Mini-graph view. While the full graph visualization gives an overview of the entire extracted data, it is not well suited for inspecting a specific entity in detail. In large graphs, finding a specific entity and reading its connections requires significant zooming and scrolling. For this reason, each Entity Detail Page includes a mini-graph that shows only the selected entity and its direct neighbors, isolated from the rest of the graph. The selected entity is highlighted in gold and placed in the center of the visualization. Entities with *incoming* relationships to the selected entity are placed on the left, while entities connected through *outgoing* are placed on the right. Entities connected in both directions appear on both sides. Edge colors are used for better understanding: green for incoming, blue for outgoing, and purple for bidirectional connections.

Figure 5 shows the mini-graph for the entity *Token*. The entity *Source Place* is connected to *Token* through two different predicates, *contains* and *marked with*. Instead of displaying two parallel edges, both predicates are combined into a single edge label. This is the edge grouping behavior described in the section above.

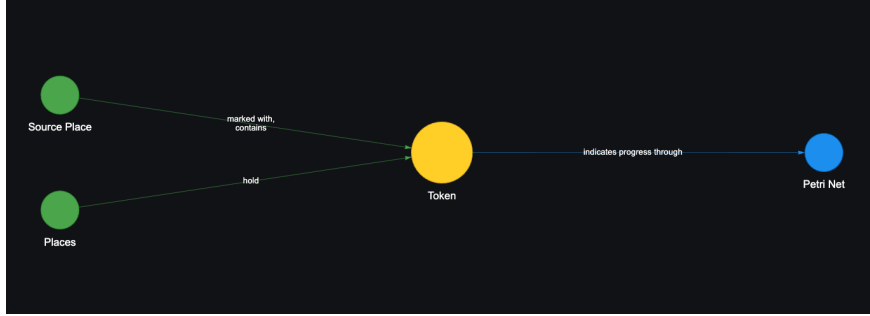


Figure 5: Mini-graph view of a single entity.

Graph statistics. The sidebar displays four graph-level statistics computed using NetworkX [8]. These are total node count, total edge count, number of connected components, and the size of the largest component. The node and edge counts show how much information was extracted from the uploaded documents, while the number of connected components reveals how fragmented the graph is. In addition to statistics, the sidebar also provides a node selector that allows the user to find and highlight any entity in the graph by name and to edit its name and description directly without navigating to the full entity detail page.

2.2 Integration Strategy

Having built the knowledge graph, the next decision was how to connect it to a RAG pipeline. Answering this question required a thorough research of existing papers on the topic. Several distinct ways to include a knowledge graph into retrieval-augmented generation were identified, each with different implications for what information reaches the LLM, at what cost, and with what trade-offs. As a result, five candidate strategies were identified from the literature [21, 7, 22] and evaluated against the requirements of this project before a choice was made.

2.2.1 Surveyed Strategies

Entity-aware Chunk Retrieval [21] extends standard RAG by adding a re-ranking step after the initial vector search. Each retrieved chunk is linked to knowledge graph entities, and the chunks are then re-ordered based on how relevant those entities are to the query. The idea is that chunks mentioning entities more closely related to the question should be prioritized over chunks that are semantically similar. While this can improve the quality of the retrieved context, it introduces a significant re-ranking layer on top of the existing pipeline. For the document sizes and query volumes in this thesis, this additional complexity is not justified.

KG Query Rewriting [21] uses the knowledge graph to expand the user query before the vector search is performed. Related entities and keywords

found in the knowledge graph are appended to the original query, producing a richer search query that is more likely to retrieve relevant chunks. The problem with this strategy is that knowledge graph information never enters the LLM prompt. The knowledge graph influences only retrieval, not what the model actually reads when generating an answer. For cross-document questions, this is not optimal, as the problem is not that the wrong chunks are retrieved, but that the relevant facts from two different documents are never in the same chunk at all.

KG Embeddings in Vector DB [21] converts knowledge graph nodes and edges into text strings, embeds them, and stores them in the vector database alongside the document chunks. The goal of this approach is simplicity, semantic search naturally returns both text chunks and KG-derived strings with no change to the retrieval pipeline. However, this approach loses the specific graph structure that makes knowledge graphs valuable. Once a relationship is serialized as a text string, the model can no longer reason over it as a structured connection between two named entities and it is treated as just another piece of text.

KG-Guided Post-RAG Refinement [7] lets the LLM generate a preliminary answer from standard RAG. The answer is then checked against the knowledge graph, and any relevant facts or corrections are passed to the model in a second call to construct a final answer. This two-step process can improve factual accuracy and reduce hallucination, but it requires two LLM calls per question. With a local 8B model taking several seconds per call, this doubles response latency.

KG-Augmented Retrieval [31] queries both the vector database and the knowledge graph for every question and combines the results into a single context that is passed to the LLM in one call. This is the most direct way to ensure that knowledge graph facts reach the model while preserving the explicit relational structure of the graph. The key challenge is controlling which KG entities are retrieved so that the prompt is not overwhelmed by loosely related information.

Table 2 summarizes the five strategies and the reasoning behind each decision.

Table 2: Overview of evaluated integration strategies.

Strategy		Core idea	Decision
Entity-aware Chunk Retrieval		Re-rank chunks by KG entity relevance	Not chosen - re-ranking complexity without clear gain
KG Query Rewriting		Expand query using KG before vector search	Not chosen - KG facts never reach the LLM
KG Embeddings in Vector DB		Embed KG nodes/edges alongside document chunks	Not chosen - loses graph structure
KG-Guided Post-RAG Refinement		Validate the RAG answer with KG in a second LLM call	Not chosen - doubles latency
KG-Augmented Retrieval		Query both sources, combine into one prompt	Chosen - KG facts reach the LLM, structure preserved

2.2.2 Chosen Approach

The goal when selecting a strategy was to find a solution that extends standard RAG in a meaningful way without requiring too much resources or too many LLM calls. KG-Augmented Retrieval [31] was chosen as it is the most direct way to bring structured knowledge graph facts into the LLM prompt while keeping the implementation simple and efficient.

The implementation combines two mechanisms that work together as a single pipeline. The first one is the retrieval step, referred to here as *Hybrid Retrieval*, inspired by the HybridRAG framework of Sarmah et al. [22]. When the user asks a question, two retrievals are performed:

1. A similarity search in the vector database that returns the top-5 most relevant document chunks.
2. The question is also analyzed to extract potential entity mentions, which are then matched against the knowledge graph to retrieve the relevant entities and their relationships.

The second mechanism is the prompt construction step, referred to here as *Dynamic Prompt Enrichment*, inspired by the KG-augmented prompting approach of Baek et al. [4] in their KAPING system. The retrieved document chunks and the knowledge graph context are combined into a single structured

prompt and given to the LLM, which is instructed to treat the document chunks as the primary source of information and to use the knowledge graph data only as complementary information. The knowledge graph thus provides additional data about the entities mentioned in the chunks without overwhelming the prompt with loosely related data.

These two mechanisms work well together as a whole. Hybrid Retrieval determines what to fetch from each source. Dynamic Prompt Enrichment determines how to present what was fetched to the LLM. How these two mechanisms are implemented in practice is described in detail in the next section.

2.3 RAG Chatbot

The second application developed in this thesis is a chatbot for answering questions about user-provided documents. Its goal is to use the data from the knowledge graph to improve answer generation.

The application was initially built as a standard RAG chatbot. The documents are split into chunks, embedded, and stored in the ChromaDB vector database [5]. ChromaDB was chosen because it runs as an embedded Python library within the application and persists on disk between sessions. When the user asks a question, the chatbot performs a similarity search on the stored embeddings and uses the retrieved document chunks to generate an answer.

After the integration strategy described in Section 2.2 was selected, the standard RAG pipeline was extended with a second retrieval mechanism based on the knowledge graph. Rather than replacing the original retrieval process, the knowledge graph retrieval builds upon it by supplying structured information about the entities mentioned in the user’s question. The complete workflow is illustrated in Figure 6.

When a question is being submitted, two retrieval processes are executed. The first follows the standard RAG path: the question is embedded using the same embedding model used during document ingestion, a cosine similarity search is performed over the chunk embeddings stored in ChromaDB, and the five most relevant document chunks are retrieved and combined into a single document context. The second retrieves information from the knowledge graph. Potential entity mentions are extracted from the question, matched against the entities stored in MongoDB using fuzzy matching, and used to retrieve the corresponding entity descriptions and relationship triples. Finally, the retrieved document chunks and the knowledge graph context are combined into a single enriched prompt, which is passed to Llama 3.1:8B to generate the final answer.

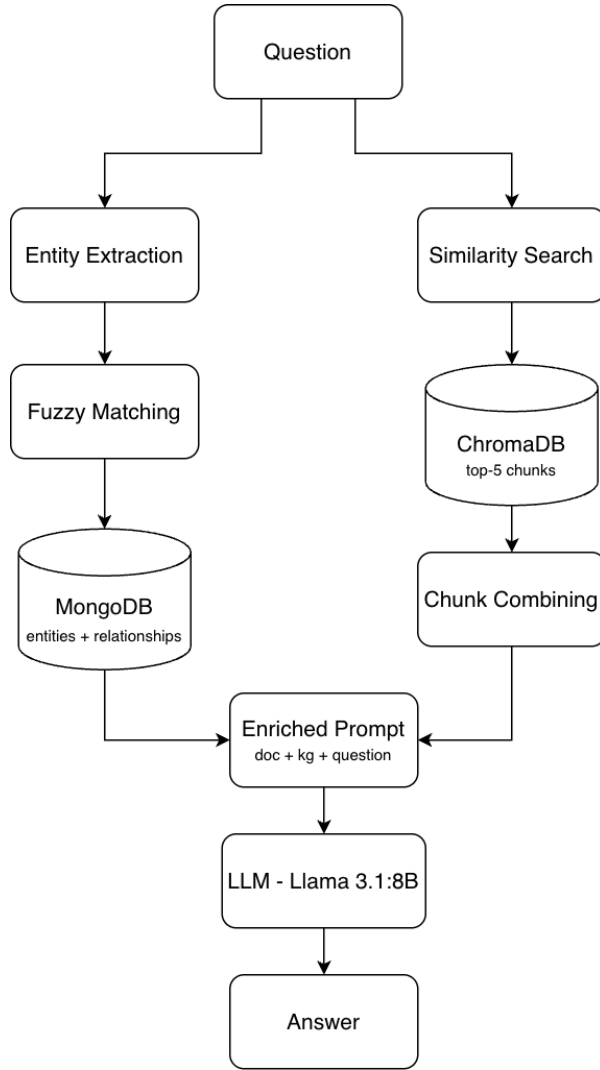


Figure 6: Chatbot workflow.

Each stage of the workflow is described in detail in the following subsections.

2.3.1 Entity Extraction

Before the knowledge graph can be queried, it is necessary to identify which entities from the user’s question might correspond to entities stored in the graph. This is done by applying Named Entity Recognition (NER) to the question text.

Named entity recognition. Named Entity Recognition is a natural language processing technique that automatically scans text and identifies mentions of real-world concepts such as people, organizations, locations, and domain-specific terms [16]. Entity linking, the task of mapping surface-form mentions to knowledge base entries, is a well-studied problem in NLP [24, 23]. If no entity candidates are found in the question, the pipeline returns an empty context and the system falls back to standard RAG without any knowledge graph augmentation.

For the NER implementation, spaCy [9] was chosen as it is a well-established and widely used open-source NLP library for Python. The model used is `en_core_web_sm`, and although relatively small, it provides efficient natural language processing capabilities, including part-of-speech tagging, dependency parsing, and named entity recognition. As the model only supports English, the current implementation is limited to processing English-language documents and queries.

Entity extraction is carried out in four stages, with each stage focusing on a different type of entity mention. The first stage extracts multi-word noun phrases using spaCy’s noun chunk detection. This is important because knowledge graph entities often consist of multiple words, and matching complete phrases generally produces more accurate results than matching individual tokens [23]. It also makes the retrieval process more robust to inconsistencies introduced during the construction of the knowledge graph. Since the Llama 3.1:8B model is relatively small, it may extract some entities as complete phrases rather than as short entity names. The second stage extracts named entities such as people, organizations, and geopolitical entities. Those are typically the most reliable candidates because the model has been specifically trained to recognize them. To improve coverage, the third stage extracts individual and proper nouns that may not have been recognized as named entities. During this process, stop words, punctuation, and tokens shorter than two characters are filtered to reduce noise. Finally, a regular expression-based fallback identifies capitalized words and phrases that were missed by the previous stages. This helps capture domain-specific terms that may not be well represented in spaCy’s general-purpose training data.

All extracted candidates are stored in a set that automatically removes duplicates generated across the extraction stages. The resulting collection is then passed to the fuzzy matching component described in the following section.

2.3.2 Fuzzy Matching

Once potential candidate entities have been extracted from the query, the next step is to identify the corresponding entities stored in the knowledge graph. A simple string match would not work reliably, because it compares characters and succeeds only when both strings are identical. For example, a query may contain *process mining* while the knowledge graph stores *Process Mining Techniques*, use an abbreviation like *Univ. Vienna* instead of *University of Vienna*, or simply contain a typo. In any of these cases, an exact match would return nothing and

the knowledge graph context would be lost.

Fuzzy matching solves this by measuring the similarity between two strings rather than checking for exact equality [24]. It assigns a similarity score between 0 and 100, where 100 means a perfect match and lower values indicate partial similarity. This makes it tolerant of typos, abbreviations, word reordering, and differences in capitalization. Because both the knowledge graph and the user queries are based on natural language, exact matches cannot always be expected. Fuzzy matching therefore increases the likelihood of retrieving the correct entities even when the wording differs.

The chatbot application uses the RapidFuzz library [3] with the WRatio scorer to compare the candidates against the entity names stored in the knowledge graph. WRatio is a similarity metric that combines several matching strategies - partial ratio, token sort ratio, and token set ratio - and returns the highest similarity score among them. This combination makes it robust to common variations such as partial matches, different word orders, abbreviations, and minor typographical errors.

The matching process is configured with the following parameters:

```
FUZZY_THRESHOLD = 75
MAX_FUZZY_MATCHES_PER_ENTITY = 2
MAX_MATCHES_PER_ENTITY = 3
```

These parameters are applied during entity matching:

```
matches = process.extract(
    entity_name,
    self._all_entity_names,
    scorer=fuzz.WRatio,
    score_cutoff=self.FUZZY_THRESHOLD,
    limit=self.MAX_FUZZY_MATCHES_PER_ENTITY
)
```

The similarity threshold of 75 was empirically selected during development to be the most optimal. Lower threshold values produced more false-positive matches, introducing unrelated entities into the prompt, whereas higher values became too restrictive and excluded relevant entities. Since there is no universally optimal threshold for entity linking [24], the selected value represents the best trade-off between precision and recall for this application.

To further limit noise, at most two fuzzy matches are received for each extracted entity candidate. This prevents any candidate term from providing too many knowledge graph entities to the prompt.

After fuzzy matching, all results are collected and deduplicated by entity name. As different candidates may match the same knowledge graph entity, deduplication avoids adding redundant context to the prompt.

2.3.3 Database retrieval

For each matched entity, two things are retrieved from MongoDB: the entity's description and all relationship triples in which the entity appears either as the

subject or the object. To reduce unnecessary database queries during retrieval, the names of all available entities are loaded from MongoDB into memory at the start of the application.

The description provides information about the entity, while the relationship triples capture its connections to other entities within the knowledge graph.

The retrieved information is formatted into a structured text block consisting of one section for the relationship triples and another for the descriptions of each matched entity together with its fuzzy match similarity score.

2.3.4 Prompt Design

Both the document context and the knowledge graph context are combined into a single prompt before being passed to Llama:

```
You are a helpful assistant. Answer the user's question
using the context below.

You have two sources of context:
1. Document Context: text chunks retrieved from the uploaded
   documents
2. Knowledge Graph Context: structured entity and
   relationship information

Instructions:
- Use Document Context as your primary source of facts
- Use Knowledge Graph Context to add relationship or entity
  information
ONLY if it directly relates to the question
- If the Knowledge Graph Context is not relevant to the
  question, ignore it
- Never invent information not present in either context
- Keep your answer to 3-4 sentences

Document Context:
{doc_context}

Knowledge Graph Context:
{kg_context}

Question: {user_input}

Answer:
```

The prompt clearly distinguishes between the retrieved document chunks and the knowledge graph data, allowing the model to identify the origin of each piece of information. It explicitly instructs the model to treat the document context as the primary source and to use the knowledge graph only as supplementary information when it is directly relevant to the user's question.

The response length is limited to three or four sentences to encourage concise, focused answers. The idea is to prevent verbose responses where the model fills space with plausible but unsupported details that are not justified by the retrieved context.

2.4 Complete System Workflow

With both applications now described, this section shows how they fit together as a complete system. Figure 7 provides an overview of the whole workflow and the steps required to use the system.

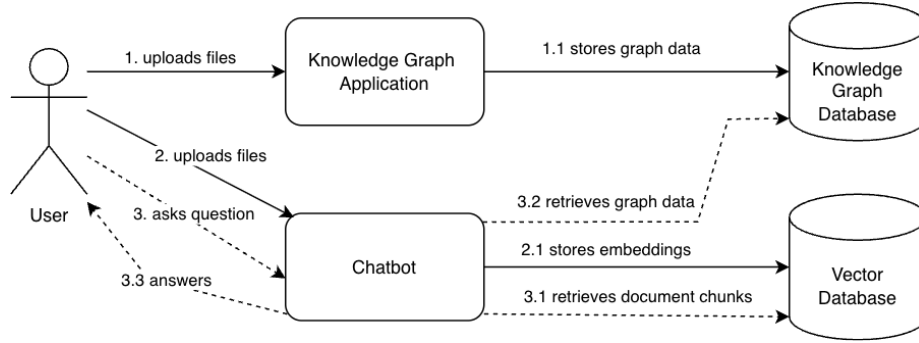


Figure 7: Complete system workflow.

The workflow begins with the user uploading documents to the Knowledge Graph Application. As described in Section 2.1, the application extracts entities and relationships from the documents and stores the resulting knowledge graph data in MongoDB. The same documents should then be uploaded to the Chatbot Application, where they are split into chunks, converted into vector embeddings, and stored in ChromaDB, 2.3. This uploading process needs to be done only once for a given set of documents. Once both databases are populated, the chatbot is ready to answer questions based on the uploaded data.

When the user asks a question, the chatbot retrieves relevant information from both databases, as explained in Section 2.3. A similarity search in ChromaDB identifies the most relevant document chunks, while named entity recognition followed by fuzzy matching against MongoDB retrieves related entities together with their descriptions and relationships. The retrieved information is combined into a single enriched prompt, which is then passed to the LLM to generate the final answer.

The user interface provides a section, which when extended shows the data and its origin, provided for the answer generation.

3 Evaluation

After implementing the proposed knowledge graph augmentation for Retrieval-Augmented Generation (RAG) system, an evaluation had to be conducted to investigate how the integration of graph information affects the chatbot’s answer generation capabilities. The goal of the evaluation was to compare the proposed knowledge graph enhanced approach with a standard RAG and to determine whether the additional knowledge graph context improves the quality of generated responses.

Rather than adopting an existing evaluation framework, the methodology used for evaluating was designed and developed as part of this thesis. Evaluating open-ended question-answering systems is challenging because there is no single metric measuring the quality of the answer. A correct answer may be expressed in many different ways, and factors such as factual accuracy, completeness, and the use of relevant context need to be considered. Traditional text-overlap metrics such as BLEU and ROUGE rely on similarity to a reference answer, but they often fail to reflect the actual quality of a response. For example, two answers containing the same information but using different wording may receive significantly different scores, while an incorrect answer that shares similar wording with the reference may still achieve a high score [7]. Manual human evaluation is generally more reliable, but it is time-consuming, expensive, and difficult to reproduce consistently.

To address these limitations, this thesis adopts the **LLM-as-a-Judge** evaluation approach introduced by Zheng et al. [30] as part of the MT-Bench framework. In this approach, a language model evaluates generated responses according to predefined criteria. Previous research has shown that LLM-based evaluations can achieve a strong correlation with human judgments while offering a more efficient and reproducible alternative for evaluating open-ended generation tasks.

The evaluation methodology evolved over the course of the project through three iterations. Each iteration introduced improvements based on limitations identified in the previous one. The following sections describe these iterations and the improvements that were done.

3.1 Evaluation Iteration 1 - Local LLM Judge

The first evaluation approach used Llama 3.1:8B as evaluation model. This is the same local language model used throughout the system for knowledge graph extraction and answer generation.

A dedicated Python script was written to automate the evaluation process. The script iterates over a predefined set of questions and, for each question, calls the chatbot twice: once only the standard RAG and once the knowledge graph-augmented pipeline. This produces two answers per question that can be compared directly. Each question was accompanied by a predefined ground truth answer describing the expected response. The evaluation model was then provided with the question, the ground truth, the generated answer and the

retrieved context. It evaluated each response separately according to the defined evaluation criteria.

The evaluation was performed using the oHA process mining paper [17] as the source document. A set of ten questions was created, covering four different question categories: factual (single-fact retrieval), multi-fact (requiring multiple sources of information), relational (questions requiring connections between entities), and causal (questions requiring explanations of underlying reasons). Particular emphasis was placed on relational questions, as these represent the type of queries where knowledge graph augmentation is expected to provide the greatest benefit by connecting entities whose relationships may not be present within a retrieved document chunk.

Both approaches were evaluated according to five criteria:

- **Correctness** - whether the information in the answer matches the provided ground truth.
- **Faithfulness** - whether the answer is supported by the context and avoids unsupported claims or hallucinations.
- **Completeness** - whether all key points in the ground truth are covered.
- **Relevance** - whether the answer addresses the given question.
- **Conciseness** - whether the answer remains focused and avoids unnecessary information or repetition.

3.1.1 Problems Identified and Changes Made

After running the evaluation a few times, three problems emerged that required changes to the setup:

Relevancy scored near maximum for both pipelines on every question. Both pipelines always answer the topic, so the criterion added no discriminating information. It was replaced with **Specificity**, which checks whether the answer provides concrete details like names, values, mechanisms. This is a criterion where KG+RAG should have a genuine advantage, since the knowledge graph injects named entity descriptions and relationship predicates into the prompt.

Conciseness constantly penalized KG+RAG answers for being longer. KG+RAG answers are usually more detailed because they synthesize two sources. Penalizing length was therefore penalizing the knowledge graph for working as intended, so conciseness was removed entirely.

Ground truths written as flowing sentences caused the judge to penalize semantically equivalent answers expressed in different words. Ground truths were rewritten as numbered key point checklists with the instructions to check meaning rather than wording. This was also supposed to help with Completeness to be easier for the judge to check how many key points are covered.

3.1.2 Limitations

Even after these improvements, the evaluation results remained inconsistent across repeated runs using the same questions. In particular, the local Llama 3.1 model frequently produced different scores for identical inputs and occasionally failed to follow the required output format. The largest inconsistencies were observed for the faithfulness criterion, which requires comparing every claim in an answer with the retrieved context. This proved to be a demanding reasoning task for an 8B model.

These observations suggested that the limitations were not caused by the evaluation criteria themselves, but by the capacity of the judge model. As a result, the local Llama-based evaluation methodology was abandoned in favor of a larger, independent language model, which is introduced in the next iteration.

3.2 Evaluation Iteration 2 - External LLM Judge

The limitations identified during the first iteration motivated replacing the local Llama 3.1:8B judge with an independent external model. Claude Sonnet [2] was selected because it provides significantly stronger reasoning capabilities and was completely independent of the system under evaluation. Using a different model as the judge also avoids the possibility of evaluating responses generated by the same model.

To support the external evaluation workflow, the Python evaluation script was extended. Rather than asking the local model to score the answers directly, the script generated a structured Markdown report containing:

- the evaluation questions,
- the ground-truth answers,
- the generated responses from both the standard RAG and knowledge graph-augmented pipelines, and
- the scoring instructions for the judge.

The original source document was attached together with this report, allowing Claude Sonnet to verify each generated answer against the source material.

The evaluation was performed using the same paper from the first iteration. The existing set of evaluation questions was reused. Each answer was assessed according to Correctness, Completeness, Faithfulness, and Specificity, with each criterion scored on a scale from 0.00 to 1.00.

3.2.1 Results

The evaluation results are presented in Table 3. The table shows the average scores across all questions for each criterion, including the difference between standard RAG and the knowledge graph-augmented approach.

Table 3: Average evaluation scores across 10 questions.

Criterion	RAG only	KG+RAG	Δ
Correctness	0.800	0.900	+0.100
Completeness	0.638	0.750	+0.113
Faithfulness	0.825	0.950	+0.125
Specificity	0.663	0.725	+0.063
Weighted overall avg	0.736	0.841	+0.105

The results show that KG+RAG outperformed slightly standard RAG on every criterion. The most significant improvements were observed for Faithfulness (+0.125) and Completeness (+0.113). These results indicate that incorporating entity data into the prompt helps the language model remain grounded in the retrieved documents while covering a larger portion of the expected answer. Similarly, the completeness gain suggests that the additional structured context helps the model cover more of the key points in a thorough answer.

To analyze the behavior of the two retrieval approaches in more detail, Table 4 breaks down the results at the individual question level.

Table 4: Per-question comparison.

Question	Type	RAG only	KG+RAG	Δ
Question 1	Factual	0.538	0.763	+0.225
Question 2	Factual	0.251	0.575	+0.326
Question 3	Factual	0.663	0.588	-0.075
Question 4	Multi-fact	0.676	0.963	+0.288
Question 5	Multi-fact	1.000	1.000	0.000
Question 6	Relational	0.800	0.900	+0.100
Question 7	Relational	0.725	0.900	+0.175
Question 8	Relational	0.975	0.975	0.000
Question 9	Causal	0.900	0.900	0.000
Question 10	Causal	0.830	0.855	+0.025

The per-question comparison shows that knowledge graph augmentation improved or maintained the answer quality of standard RAG on nine of the ten evaluation questions. The largest improvements were observed for Question 2 (+0.326) and Question 4 (+0.288), where the additional entity descriptions and relationships provided useful context that was not sufficiently captured through semantic document retrieval alone.

The only case in which the enhanced system performed worse than standard RAG was Question 3 (-0.075), a factual question. In it, an incorrectly matched entity introduced irrelevant context into the prompt, slightly distracting the model from the correct answer.

Overall, the evaluation demonstrates that knowledge graph augmentation improves answer quality for a single-document collection. However, the observed improvement remains relatively modest, with an overall increase of $+0.105$. One possible explanation for this is that the evaluation was performed using only a single document consisting of 11 pages. In such case, standard RAG is already capable of retrieving most of the relevant information through semantic similarity alone.

The primary motivation for integrating a knowledge graph, however, is not to improve simple single-document retrieval, but to enable connections between related information distributed across multiple documents. This motivated the third evaluation iteration.

3.3 Evaluation Iteration 3 - Multi-Document Evaluation

For the third iteration, two additional papers were added to both the Knowledge Graph Application and the Chatbot Application. The first paper, *Process Mining: A 360 Degree Overview* by van der Aalst [28], provides an overview of the three main categories of process mining: process discovery, conformance checking, and process enhancement. The second paper by Koschmider et al. [13] discusses the application of process mining to unstructured data sources. Together with the original oHA paper [17], these documents form a small document collection covering related aspects of process mining.

The question set was expanded from 10 to 19 questions and a new **cross-document** category was added, consisting of questions that require information from two or more papers and therefore cannot be answered completely by relying on a single document only.

The motivation for this evaluation is based on the main difference between semantic retrieval and knowledge graph retrieval. Standard RAG retrieves document chunks independently according to semantic similarity. As a result, answering a question that requires information from multiple documents depends on retrieving all relevant chunks at the same time, which becomes increasingly difficult as the document collection grows. In contrast, the knowledge graph stores entities and their relationships independently of the original documents. This makes it possible to retrieve related concepts from different documents through shared entities, even when the corresponding text chunks are not retrieved together.

3.3.1 Results

The primary objective of this iteration was to evaluate whether knowledge graph augmentation improves retrieval performance for questions that require combining information across multiple documents. Table 5 summarizes the weighted

evaluation scores obtained for the six additional cross-document questions.

Table 5: Performance comparison for cross-document questions.

Question	Papers	RAG	KG+RAG	Δ
Question 12	oHA + vdA	0.238	0.776	+0.538
Question 13	oHA + K	0.213	0.348	+0.135
Question 14	oHA + vdA + K	0.288	0.333	+0.045
Question 15	oHA + K	0.603	0.698	+0.095
Question 16	oHA + vdA	0.603	0.698	+0.095
Question 17	oHA + vdA	0.308	0.358	+0.050
Weighted overall avg		0.375	0.535	+0.160

The results show that KG+RAG achieved higher scores than standard RAG for all six cross-document questions, with an average improvement of +0.160. This represents a larger improvement than the one observed during the single-document evaluation (+0.105), suggesting that the benefits of knowledge graph context increase when relevant information is scattered across multiple sources.

3.3.2 Cross-Document Analysis

The largest improvement was observed for Question 12 (+0.538), which required combining information from the oHA [17] paper and the work of van der Aalst [28]. Standard RAG achieved a relatively low score of 0.238, because the relevant information was distributed across two different documents and the required document chunks were not retrieved together in the top-5 results.

The remaining cross-document questions also showed improvements with the KG+RAG approach, although with smaller gains. For Questions 13, 14, and 17, both approaches struggled to produce highly accurate answers. This suggests that some of the required relationships were either not extracted during knowledge graph construction or were not successfully retrieved through the entity linking and fuzzy matching process. These cases therefore highlight limitations of the current extraction and retrieval pipeline rather than limitations of the knowledge graph concept itself.

3.4 Discussion

The results of the evaluation reveal that standard RAG and knowledge graph enhanced RAG have different strengths depending on the type of question being answered. For questions where the required information is contained within a single document, standard RAG already performs well. Consequently, adding

knowledge graph context provides only modest improvements and may occasionally introduce irrelevant context when entity matching is imperfect.

The advantages of knowledge graph augmentation become more evident when information must be combined from multiple documents. Because entities and their relationships are stored independently of the documents, the knowledge graph can retrieve connections that would otherwise require multiple semantically unrelated chunks to be retrieved simultaneously. This cannot be reproduced through semantic similarity alone.

Overall, the evaluation shows that knowledge graph augmentation does not significantly improve every type of question equally. Instead, its greatest strengths become evident in scenarios requiring cross-document reasoning. The results also show that the effectiveness of the approach depends strongly on the quality of the underlying knowledge graph. Incorrect or overly permissive fuzzy matches may introduce unrelated entities into the prompt, reducing answer quality. Future improvements should therefore focus on more accurate named entity recognition, improved entity disambiguation, and more robust entity linking strategies to further increase the effectiveness of the proposed approach.

4 Conclusion and Future Work

4.1 Conclusion

This thesis presented the design and implementation of a fully local knowledge graph creating application, which was integrated with a Retrieval-Augmented Generation (RAG) chatbot and evaluated.

To achieve this, two different applications were developed. The Knowledge Graph Application provides an automated pipeline for extracting entities, descriptions, and subject-predicate-object relationships from uploaded documents and storing the resulting knowledge graph in MongoDB.

The Chatbot Application integrates the generated knowledge graph into a RAG pipeline using the selected *Knowledge Graph Augmented Retrieval* strategy. This approach was chosen after evaluating different integration methods. The implemented pipeline combines semantic document retrieval with knowledge graph retrieval. Relevant document chunks are retrieved through similarity search, while additional entity information and relationships are obtained through named entity recognition and entity linking. Both sources are then combined in one prompt and passed to the LLMs for answer generation.

The developed system was evaluated through a custom evaluation methodology conducted in three iterations. The evaluation showed that knowledge graph augmentation provides in some cases a slight improvement for questions that can already be answered from a single document. However, its advantages become more apparent for questions requiring information from multiple documents, where explicit entity relationships help connect information that is difficult to retrieve through semantic search alone.

An additional contribution of this thesis is to demonstrate that knowledge

graph augmentation can improve retrieval-augmented generation even when using a relatively small local language model, namely Llama 3.1:8B. This is particularly relevant for users who wish to run such LLM-based systems on consumer GPUs.

Overall, this thesis demonstrates that knowledge graph augmentation is particularly valuable when answering questions that require connecting information across different sources. By combining semantic retrieval with structured knowledge representation, the proposed approach extends the capabilities of traditional RAG systems and provides a foundation for more context-aware question answering over domain-specific document collections.

4.2 Limitations

Although the proposed system achieved encouraging results, several limitations should be considered when interpreting the findings.

First, the evaluation was conducted on a relatively small collection of three process mining papers containing 19 evaluation questions. Although this was sufficient to demonstrate the feasibility of the proposed approach, it is not enough to present significant results for general conclusions.

Second, the quality of the knowledge graph depends on the quality of the entity and relationship extraction process. Since extraction is performed using Llama 3.1:8B, extraction errors such as missing relationships, inconsistent entity names, or incorrectly identified entities can propagate through the retrieval pipeline and reduce answer quality.

Another limitation is the entity linking approach. The current implementation relies on fuzzy string matching with a fixed similarity threshold to identify corresponding entities in the knowledge graph. Although this approach proved effective in most cases, occasional incorrect matches introduced unrelated entities into the prompt, particularly for simple factual questions.

Finally, the current implementation supports only English-language documents because the spaCy model used for named entity recognition is limited to English. Extending the system to multilingual documents would require replacing or extending the current entity extraction pipeline.

4.3 Future Work

Several opportunities for further improvement were identified during the development and evaluation.

The most impactful improvement would be improving the precision of the entity linking stage. The current implementation relies on fuzzy string matching with a fixed similarity threshold. Future versions could incorporate context-aware entity disambiguation methods that use both the user query and the surrounding document context when selecting the most appropriate entity match. This would reduce incorrect matches while preserving the benefits of knowledge graph retrieval.

The current implementation uses Llama 3.1:8B for both knowledge graph construction and answer generation. While this model provides good results for consumer hardware with limited resources, its limited reasoning capabilities can affect the quality of entity and relationship extraction, resulting in incomplete or inaccurate knowledge graphs. Larger language models could improve extraction quality and make better use of graph information during answer generation, particularly for complex queries requiring multi-step relational reasoning.

Another possible improvement is the introduction of adaptive retrieval. The evaluation showed that knowledge graph augmentation provides the biggest benefit for cross-document questions, while simple factual questions often require little additional graph context. Identifying the type of query and selectively enabling knowledge graph retrieval could improve both retrieval efficiency and answer quality.

From a usability perspective, the current system rebuilds entity descriptions from scratch whenever new documents are processed. Supporting incremental updates would allow users to extend an existing knowledge graph without re-processing previous documents, reducing processing time for larger collections of documents.

Finally, a larger-scale evaluation would provide stronger evidence for the effectiveness of the proposed approach. Future experiments could include larger document collections from different domains, as well as a broader and more diverse set of evaluation questions.

Source Code

The source code developed as part of this thesis is available at:

Knowledge Graph Application:

https://github.com/DragomirMar/Knowledge_Graph

Chatbot Application:

<https://github.com/DragomirMar/ChatBot>

References

- [1] ANDRÉS, D. Issue #102 — knowledge graphs to make RAG smarter. Machine Learning Pills (Substack newsletter), July 2025. Retrieved 2025 from <https://mlpills.substack.com/p/issue-102-knowledge-graphs-to-make>.
- [2] ANTHROPIC. Claude sonnet: Model card. <https://www.anthropic.com/claude>, 2024. Accessed 2025.
- [3] BACHMANN, M. RapidFuzz: A fast string matching library for Python and C++. <https://github.com/rapidfuzz/RapidFuzz>, 2021. MIT License, version 3.x.
- [4] BAEK, J., AJI, A. F., AND SAFFARI, A. Knowledge-augmented language model prompting for zero-shot knowledge graph question answering. In *Proceedings of the NLRSE Workshop at ACL 2023* (2023). arXiv:2306.04136.
- [5] CHROMA TEAM. ChromaDB: The AI-native open-source embedding database. <https://www.trychroma.com>, 2023. Open-source embedded vector database.
- [6] EDGE, D., TRINH, H., CHENG, N., BRADLEY, J., CHAO, A., MODY, A., TRUITT, S., AND LARSON, J. From local to global: A graph RAG approach to query-focused summarization. arXiv:2404.16130.
- [7] GAO, Y., XIONG, Y., GAO, X., JIA, K., PAN, J., BI, Y., DAI, Y., SUN, J., WANG, M., AND WANG, H. Retrieval-augmented generation for large language models: A survey. arXiv:2312.10997.
- [8] HAGBERG, A. A., SCHULT, D. A., AND SWART, P. J. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)* (2008), G. Varoquaux, T. Vaught, and J. Millman, Eds., pp. 11–15.
- [9] HONNIBAL, M., MONTANI, I., VAN LANDEGHEM, S., AND BOYD, A. spaCy: Industrial-strength natural language processing in python, 2020. DOI: 10.5281/zenodo.1212303. Available: <https://spacy.io>.
- [10] JAIN, T. Evaluate RAG pipeline using HuggingFace open source models. Hugging Face Community Blog, June 2024. Accessed 2025.
- [11] JI, S., PAN, S., CAMBRIA, E., MARTTINEN, P., AND YU, P. S. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Transactions on Neural Networks and Learning Systems* 33, 2 (2021), 494–514.

- [12] KOMMINENI, V. K., KÖNIG-RIES, B., AND SUBHASHINI, S. LLMs for knowledge graph construction and reasoning: Recent capabilities and future opportunities. In *Proceedings of the ACL 2024 Workshop on Knowledge Graphs and Large Language Models (KaLLM)* (2024). arXiv:2305.13168.
- [13] KOSCHMIDER, A., ALEKNONYTE-RESCH, M., FONGER, F., KUHN, L., SCHUHMAN, E., AND REHSE, J.-R. Process mining for unstructured data: Challenges and research directions. arXiv:2401.13677.
- [14] LANGCHAIN TEAM. LangChain: Building applications with LLMs through composability. <https://github.com/langchain-ai/langchain>, 2023. Open-source framework, version 0.1.x.
- [15] LEWIS, P., PEREZ, E., PIKTUS, A., PETRONI, F., KARPUKHIN, V., GOYAL, N., KÜTTLER, H., LEWIS, M., YIH, W.-T., ROCKTÄSCHEL, T., RIEDEL, S., AND KIELA, D. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems* (2020), vol. 33. arXiv:2005.11401.
- [16] LI, J., SUN, A., HAN, J., AND LI, C. A survey on deep learning for named entity recognition. *IEEE Transactions on Knowledge and Data Engineering* 34, 1 (2022), 50–70. arXiv:1812.09449.
- [17] LUX, M., AND RINDERLE-MA, S. Problems and challenges of applying process mining to a tourist information system. In *Proceedings of the BPM 2017 Industry Track* (2017). BPM17industry01.
- [18] MONGODB INC. MongoDB: A document-oriented NoSQL database. <https://www.mongodb.com>, 2009. Open-source document database, version 7.x.
- [19] OLLAMA TEAM. Ollama: Run large language models locally. <https://ollama.com>, 2023. Open-source local model serving platform.
- [20] PAN, S., LUO, L., WANG, Y., CHEN, C., WANG, J., AND WU, X. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering* (2024). arXiv:2306.08302.
- [21] PENG, B., ZHU, Y., LIU, Y., BO, X., SHI, H., HONG, C., ZHANG, Y., AND TANG, S. Graph retrieval-augmented generation: A survey. *ACM Transactions on Information Systems* (2024). arXiv:2408.08921.
- [22] SARMAH, B., HALL, B., RAO, R., PATEL, S., PASQUALI, S., AND MEHTA, D. HybridRAG: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction. In *International Conference on AI in Finance* (2024). arXiv:2408.04948.

- [23] SEVGILI, Ö., SHELMANOV, A., ARKHIPOV, M., PANCHENKO, A., AND BIEMANN, C. Neural entity linking: A survey of models based on deep learning. *Semantic Web* (2022). DOI: 10.3233/SW-222986.
- [24] SHEN, W., WANG, J., AND HAN, J. Entity linking with a knowledge base: Issues, techniques, and solutions. *IEEE Transactions on Knowledge and Data Engineering* 27, 2 (2015), 443–460.
- [25] SOMAN, K., ROSE, P. W., MORRIS, J. H., AKBAS, R. E., SMITH, B., PEETOOM, B., VILLOUTA-REYES, C., CERONO, G., SHI, Y., RIZK-JACKSON, A., AND CAVANAUGH, S. Biomedical knowledge graph-enhanced prompt generation for large language models. arXiv:2405.12035.
- [26] STREAMLIT INC. Streamlit: The fastest way to build data apps. <https://streamlit.io>, 2019. Open-source Python UI framework.
- [27] TONMOY, S. M. T. I., ZAMAN, S. M. M., JAIN, V., RANI, A., RAWTE, V., CHADHA, A., AND DAS, A. A comprehensive survey of hallucination mitigation techniques in large language models. arXiv:2401.01313.
- [28] VAN DER AALST, W. M. P. Process mining: A 360 degree overview. In *Process Mining Handbook*, W. M. P. van der Aalst and J. Carmona, Eds., vol. 448 of *Lecture Notes in Business Information Processing*. Springer, 2022. Available: <https://www.vdaalst.com/publications/p1329.pdf>.
- [29] XU, C., TAN, W., MIN, B., AND CHEN, M. Large language models for generative information extraction: A survey. arXiv:2312.17617.
- [30] ZHENG, L., CHIANG, W.-L., SHENG, Y., ZHUANG, S., WU, Z., ZHUANG, Y., LIN, Z., LI, Z., LI, D., XING, E. P., ZHANG, H., GONZALEZ, J. E., AND STOICA, I. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems* (2023). arXiv:2306.05685.
- [31] ZHU, X., XIE, Y., LIU, Y., LI, Y., AND HU, W. Knowledge graph-guided retrieval augmented generation. arXiv:2502.06864.